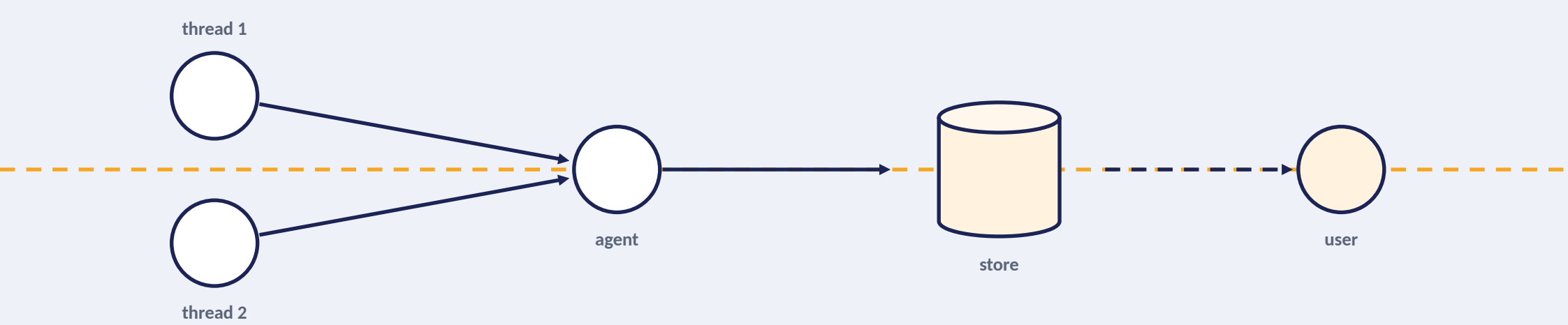
LANGGRAPH · CHAPTER 10 · FLASHCARDS

Long-Term Memory

Facts that follow a user

9 code cards on the Store, two identities and safe writes



The thread and the user

A checkpointer remembers one conversation; a Store remembers a person across all of them

- One user owns many threads, each with its own history, and all of them share one profile.
- Scope and durability are separate choices: InMemoryStore is long-term in scope, yet dies on restart.

TEXT

```
                  process RAM only          a database
                +-------------------------+-------------------------+
one thread      |  MemorySaver            |  PostgresSaver          |
(thread_id)     |  Chapter 9              |  Chapter 9              |
                +-------------------------+-------------------------+
every thread    |  InMemoryStore          |  PostgresStore          |
of one user     |  long-term scope,       |  long-term scope,       |
(user_id)       |  gone on restart        |  survives a restart     |
                +-------------------------+-------------------------+

checkpointer + thread_id  ->  "where is this conversation?"
Store + user_id           ->  "what should future conversations know?"
```

namespace + key → value

The namespace tuple is a folder, the key a filename, and the value a plain dict

- put writes or overwrites, get fetches one exact item, search lists a namespace. No model, no graph.
- Put the user_id in the namespace: that tuple is what keeps one user's memories away from another's.

PYTHON

```python
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()
namespace = ("walid", "memories")     # (user_id, "memories"): a folder per user

# put(namespace, key, value): create or update one memory
store.put(namespace, "profile", {"name": "Walid", "role": "software engineer",
                                 "preference": "concise explanations"})
store.put(namespace, "learning_goal", {"topic": "LangGraph long-term memory"})

profile = store.get(namespace, "profile")       # one exact memory, or None
print(profile.value)
# {'name': 'Walid', 'role': 'software engineer', 'preference': ...}

for item in store.search(namespace):            # the memories in the folder
    print(f"- {item.key}: {item.value}")
# - profile: {'name': 'Walid', ...}
# - learning_goal: {'topic': 'LangGraph long-term memory'}
```

get returns an Item; put replaces it

Reading is two steps, item then .value; writing the same key swaps in the whole new dict

- An Item carries namespace, key, value, created_at and updated_at, plus a score for semantic search.
- Plain text still goes in a dict, {"memory": text}; delete(namespace, key) removes one memory.

PYTHON

```python
item = store.get(("walid", "memories"), "profile")
item.namespace, item.key     # ('walid', 'memories'), 'profile'
item.value                   # the dict you saved
item.updated_at              # a datetime; so is created_at

store.get(("guest", "memories"), "profile")     # None: nothing there

# The same namespace + key UPDATES the entry and replaces the WHOLE value
store.put(("walid", "memories"), "profile", {"role": "engineering manager"})
store.get(("walid", "memories"), "profile").value
# {'role': 'engineering manager'}       <- name and preference are gone

# Plain text: wrap it in a dict, read it back by its field
store.put(("memory", "walid"), "user_details", {"memory": profile_text})
store.get(("memory", "walid"), "user_details").value["memory"]

store.delete(("walid", "memories"), "learning_goal")   # corrections, removals
```

Two memories, two identities

thread_id in config picks the conversation; user_id in context picks the person's memory

- Both memories attach at compile time, just as the checkpointer did in Chapter 9.
- Change the thread_id: a fresh chat with the same profile. Change the user_id: a different person.

PYTHON

```python
from dataclasses import dataclass
from langgraph.runtime import Runtime

@dataclass
class Context:
    user_id: str          # identifies a PERSON, not a conversation

builder = StateGraph(MessagesState, context_schema=Context)
builder.add_node("chat", chat)                    # reads the profile
builder.add_node("update_memory", update_memory)  # writes new facts back
# START -> chat -> update_memory -> END

graph = builder.compile(
    checkpointer=MemorySaver(),   # short-term: message history per thread_id
    store=InMemoryStore(),        # long-term: user facts per namespace
)

config  = {"configurable": {"thread_id": "walid-chat-1"}}   # the conversation
context = Context(user_id="walid")                          # the user's memory
graph.invoke({"messages": [{"role": "user", "content": "Hi"}]},
             config, context=context)
```

Runtime brings the Store and the context

A node that takes runtime: Runtime[Context] gets the compiled Store and your user_id

- runtime.context.user_id is the id you passed; runtime.store is the Store attached at compile().
- The profile goes into the system message, so the model answers from it and the restored thread.

PYTHON

```python
def chat(state: MessagesState, runtime: Runtime[Context]) -> dict:
    namespace = ("memory", runtime.context.user_id)      # this user's folder
    saved = runtime.store.get(namespace, "user_details")
    profile = (saved.value["memory"] if saved          # get() returns an Item
               else "No existing user details found.")

    system = SystemMessage(
        "You are a helpful assistant with memory. Personalize your answer "
        "only with facts found in the saved user details.\n\n"
        f"Saved user details:\n{profile}"
    )
    response = model.invoke([system, *state["messages"]])
    return {"messages": [response]}

# Forget context= at invoke and runtime.context is None:
# AttributeError: 'NoneType' object has no attribute 'user_id'
# Forget store= at compile and runtime.store is None:
# AttributeError: 'NoneType' object has no attribute 'get'
```

Extract, merge, then put

Extract with structured output from the latest human message, then merge field by field

- Read only the newest HumanMessage, so the model can't bank claims the assistant invented.
- Start from the stored profile: a new role keeps old preferences, and the newest explicit value wins.

PYTHON

```python
class MemoryUpdate(BaseModel):     # only facts stated in the latest message
    name: str | None = None
    role: str | None = None
    preferences: list[str] = Field(default_factory=list)

def update_memory(state: MessagesState, runtime: Runtime[Context]) -> dict:
    namespace = (runtime.context.user_id, "memories")
    existing_item = runtime.store.get(namespace, "profile")
    existing = dict(existing_item.value) if existing_item else {}
    latest = next(m for m in reversed(state["messages"])
                  if isinstance(m, HumanMessage))        # never the AI's words
    update = memory_extractor.invoke([EXTRACT_PROMPT, latest])

    merged = dict(existing)                          # start from what we know
    if update.name: merged["name"] = update.name
    if update.role: merged["role"] = update.role  # the newest value wins
    prefs = [*merged.get("preferences", []), *update.preferences]
    if prefs: merged["preferences"] = list(dict.fromkeys(prefs))   # no dups
    if merged != existing:
        runtime.store.put(namespace, "profile", merged)
    return {}
```

New thread, same user, same profile

The checkpointer looks up thread_id, the Store looks up user_id: two lookups, two scopes

- walid-chat-2 starts with no old messages, yet chat reads the profile that thread 1 saved.
- The namespace is the whole security boundary: take user_id from the login session, never from input.

PYTHON

```python
walid = Context(user_id="walid")

run_thread(graph, "walid-chat-1", walid, "Hi, my name is Walid. I am a ...")
# update_memory saves ("walid", "memories") / "profile"

run_thread(graph, "walid-chat-2", walid, "This is a new chat. I am now an ...")
# checkpointer: thread "walid-chat-2" is EMPTY (a new conversation)
# Store: ("walid", "memories") / "profile" is FOUND (the same person)
# -> the reply knows the name; update_memory merges in the new role

run_thread(graph, "guest-chat-1", Context(user_id="guest"),
           "What do you remember about me?")
# ("guest", "memories") is empty: no Walid memory at all
```

Long-term scope is not durability

PostgresStore keeps the same API and the profile survives the process that wrote it

- The writer exits before the reader starts, so the profile can only come back from PostgreSQL.
- PostgresStore is not PostgresSaver: facts by namespace versus checkpoints by thread_id.

PYTHON

```python
from langgraph.store.postgres import PostgresStore

# Process A (01_save_profile.py): save one profile, then EXIT
with PostgresStore.from_conn_string(DB_URI) as store:
    # store.setup() runs ONCE, in 00_setup_tables.py, to create the tables
    store.put(("memory", "user-1"), "user_details",
              {"name": "Walid", "role": "Engineering manager",
               "preferences": ["concise explanations"]})

# Process B (02_read_profile.py): a fresh Python process reads it back
with PostgresStore.from_conn_string(DB_URI) as store:
    item = store.get(("memory", "user-1"), "user_details")
print(item.value)    # {'name': 'Walid', ...}: from PostgreSQL, not RAM

# A production graph compiles with both:
#   compile(checkpointer=<PostgresSaver>, store=<PostgresStore>)
```

Store traps that raise no error

search() stops at 10 items and matches by prefix; put() drops any field you leave out

- search(namespace) returns 10 items by default: pass limit= when you really mean every memory.
- A shorter namespace is a prefix: search(("memory",)) returns every user's ("memory", id) folder.

PYTHON

```python
ns = ("walid", "memories")
for i in range(15):
    store.put(ns, f"note-{i}", {"i": i})
len(store.search(ns))                  # 10, not 15
len(store.search(ns, limit=100))       # 15

store.put(("memory", "user-1"), "user_details", {"memory": "a"})
store.put(("memory", "user-2"), "user_details", {"memory": "b"})
[item.namespace for item in store.search(("memory",))]
# [('memory', 'user-1'), ('memory', 'user-2')]     <- both users

# put() replaces the whole value: merge first, then write
old = store.get(ns, "profile")
store.put(ns, "profile", {**(old.value if old else {}), "role": "manager"})

# Checked on langgraph 1.2.6 (InMemoryStore)
```